

RICOSTRUZIONE 3D **DI UN AMBIENTE AGRICOLO CON OPEN3D**

STUDENTESSE:

Butticé Giada
Placchi Fabiana Maria

PROFESSORE:

Prof. Daniele Meli

ANNO ACCADEMICO:

2025/2026



IL PROBLEMA

Perché la ricostruzione 3D?

La ricostruzione 3D di un ambiente agricolo permette la creazione di un ***digital twin*** del campo.

Questo modello digitale può supportare:

- Monitoraggio delle colture
- Stima della biomassa
- Analisi della struttura della vegetazione
- Pianificazione degli interventi agricoli.

OBIETTIVO PROGETTO:

Ottenere una ricostruzione tridimensionale di un ambiente agricolo utilizzando un robot mobile dotato di una camera orientata verso il campo.

PIPELINE

Abbiamo sviluppato una pipeline composta da due fasi principali:

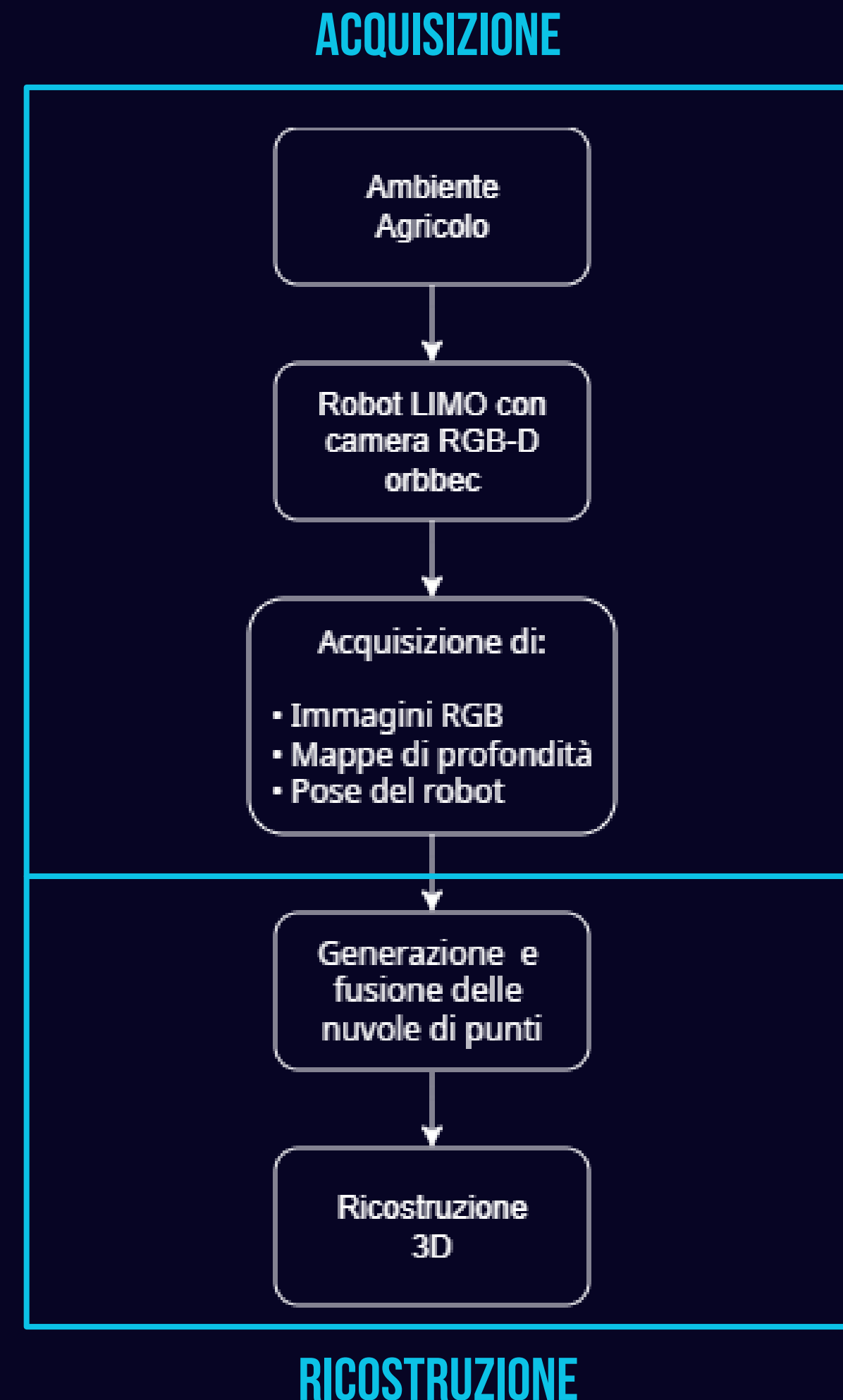
1. L'acquisizione dei dati
2. La ricostruzione 3D dell'ambiente.

Il robot che monta una camera RGB-D orbbec, acquisisce informazioni sulla scena:

- un'immagine RGB
- una mappa di profondità
- le pose del robot.

Per ogni acquisizione viene generata una **nuvola di punti**.

Di seguito tutte le nuvole di punti vengono fuse tra loro ottenendo la ricostruzione 3D completa della scena.

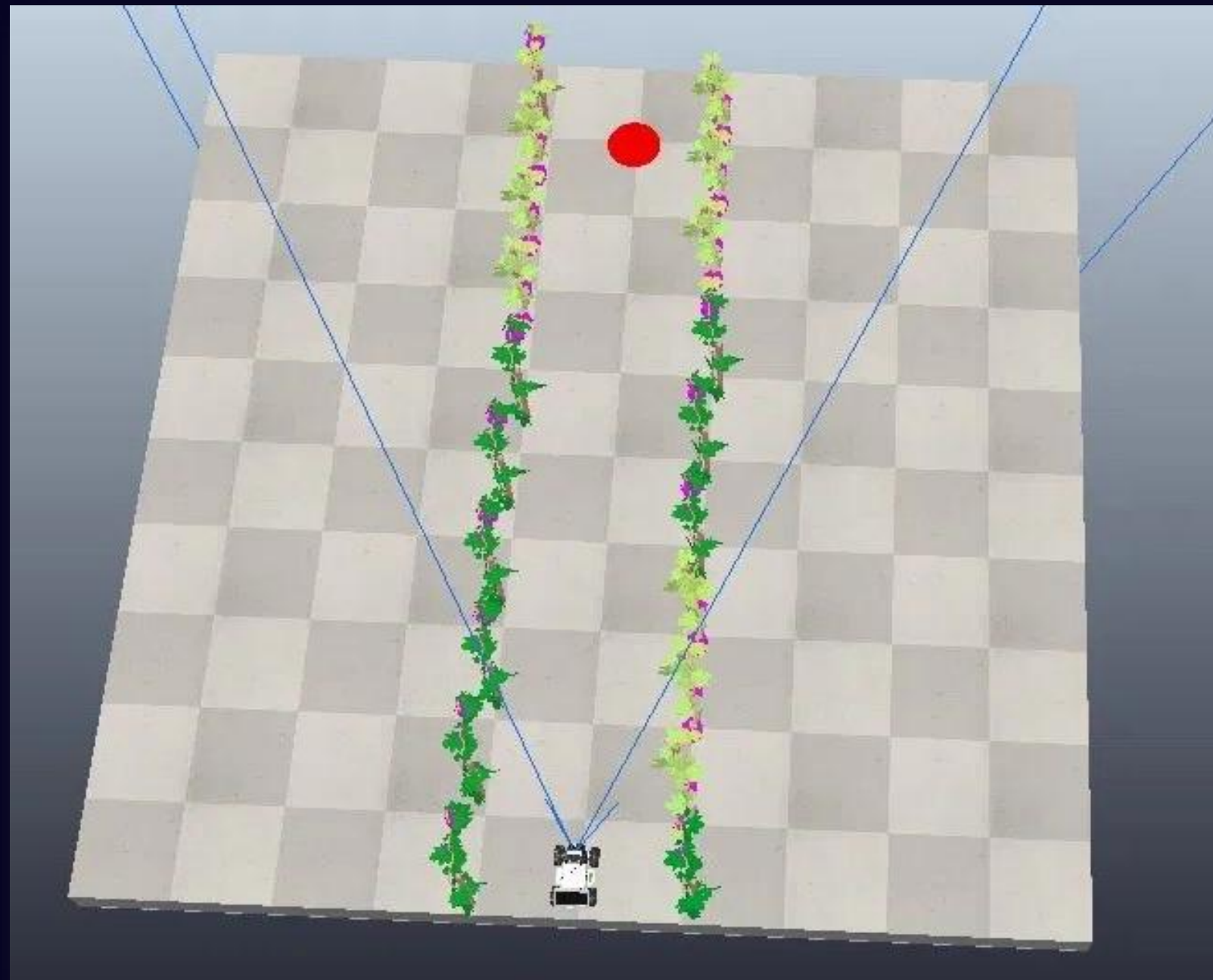


ACQUISIZIONE DEI DATI



CoppeliaSim
from the creators of V-REP

ROS.org



Abbiamo sfruttato la scena di **CoppeliaSim** già sviluppata nel secondo laboratorio del corso. Questa simula un campo agricolo che presenta due filari paralleli, con all'interno il robot LIMO in movimento.

Per fare questo è stato sviluppato un nodo ROS dedicato, denominato *data collector*, che svolge due funzioni principali:

- **Gestione del moto del robot:** attraverso un control loop, il sistema calcola la direzione del punto target, allinea il robot rispetto a questo target e lo fa avanzare ad una velocità di 0,4 m/s fino al raggiungimento della destinazione.
- **Raccolta dei dati:** durante la percorrenza del filare, il nodo genera il dataset necessario per la successiva fase di ricostruzione 3D.

ACQUISIZIONE DEI DATI

Il nodo ROS genera un dataset, denominato *reconstruction_data*, organizzato nelle seguenti componenti:

reconstruction_data/

./color/

./depth/

./poses/

./camera_intrinsics.json



CoppeliaSim
from the creators of V-REP

ROS.org

- **Color:** contiene le immagini RGB acquisite dalla camera RGB-D Orbbec.
- **Depth:** contiene le mappe di profondità provenienti dalla camera RGB-D.
- **Poses:** contiene le matrici di trasformazione che descrive la posizione e l'orientamento del robot per ognuna delle acquisizioni.
- **Camera Intrinsics:** contiene i parametri intrinseci della camera RGB-D, costituiti dalla risoluzione dell'immagine (width, height), dalle lunghezze focali (f_x , f_y) e dalle coordinate del punto principale (c_x , c_y).

Il control loop del robot viene eseguito con una frequenza di 20 Hz (0,05s), mentre il salvataggio dei dati avviene ogni tre iterazioni del control loop, corrispondenti ad un salvataggio nel dataset ogni 0,15s.

RICOSTRUZIONE 3D



```
DATASET = "reconstruction_data"    # dataset in cui sono salvati i dati per ricostruire la scena
OUT_FILE = "fusion_final.ply"      # finestra output di open3d

# Conta i frame presenti nel dataset (num rgb = num depth)
N_FRAMES = len([f for f in os.listdir(os.path.join(DATASET, "color"))])

# Caricamento dei parametri della camera salvati durante la fase di acquisizione
with open(os.path.join(DATASET, "camera_intrinsics.json"), "r") as f:
    K = json.load(f)

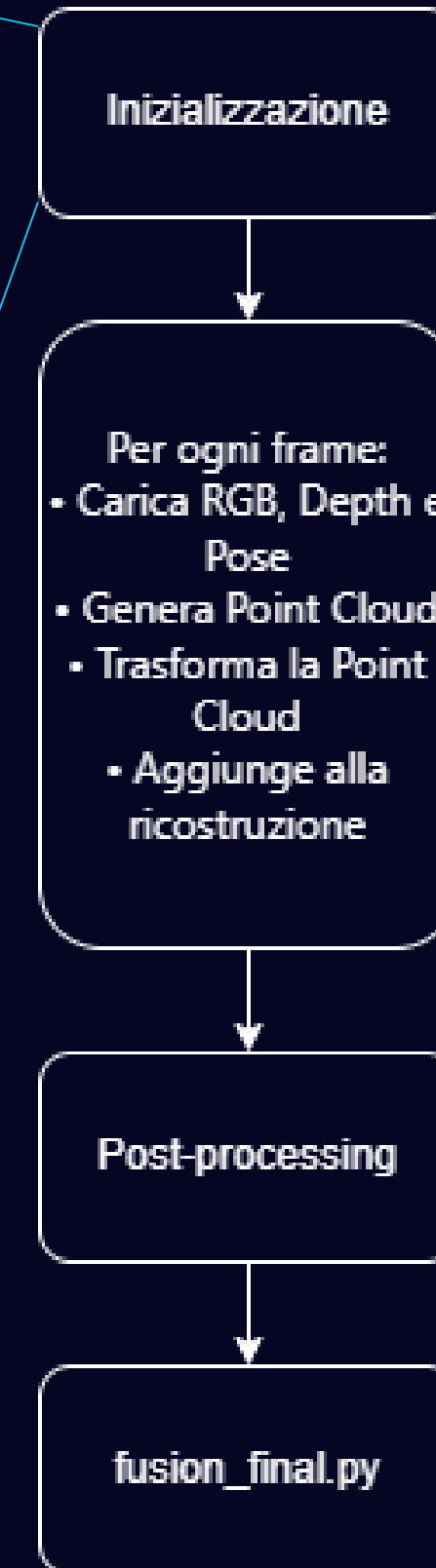
# Creazione del modello pinhole per la proiezione dei pixel nello spazio 3D
intrinsic = o3d.camera.PinholeCameraIntrinsic(
    K["width"], K["height"],
    K["fx"],    K["fy"],
    K["cx"],    K["cy"]
)

# Trasformazione dal sistema di riferimento della camera al sistema di riferimento del robot

# Mappatura
# X_robot (Avanti)  = Z_camera (Profondità)
# Y_robot (Sinistra) = -X_camera (Destra invertita)
# Z_robot (Alto)    = -Y_camera (Basso invertito)

T_cam_to_robot = np.array([
    [ 0.0,  0.0,  1.0,  0.0],
    [-1.0,  0.0,  0.0,  0.0],
    [ 0.0, -1.0,  0.0,  0.0],
    [ 0.0,  0.0,  0.0,  1.0]
])

# Nuvola di punti globale contiene la fusione di tutti i frame
global_cloud = o3d.geometry.PointCloud()
```



RICOSTRUZIONE 3D



```
for i in range(N_FRAMES):

    color_path = os.path.join(DATASET, "color", f"frame_{i:05d}.png")
    depth_path = os.path.join(DATASET, "depth", f"frame_{i:05d}.npy")
    pose_path = os.path.join(DATASET, "poses", f"frame_{i:05d}.txt")

    if not os.path.exists(color_path):
        continue

    print(f"\rFrame {i+1}/{N_FRAMES}", end="", flush=True)

    # Caricamento dell' RGB e della depth
    color = o3d.io.read_image(color_path)
    depth_np = np.load(depth_path)
    depth = o3d.geometry.Image(depth_np.astype(np.float32))

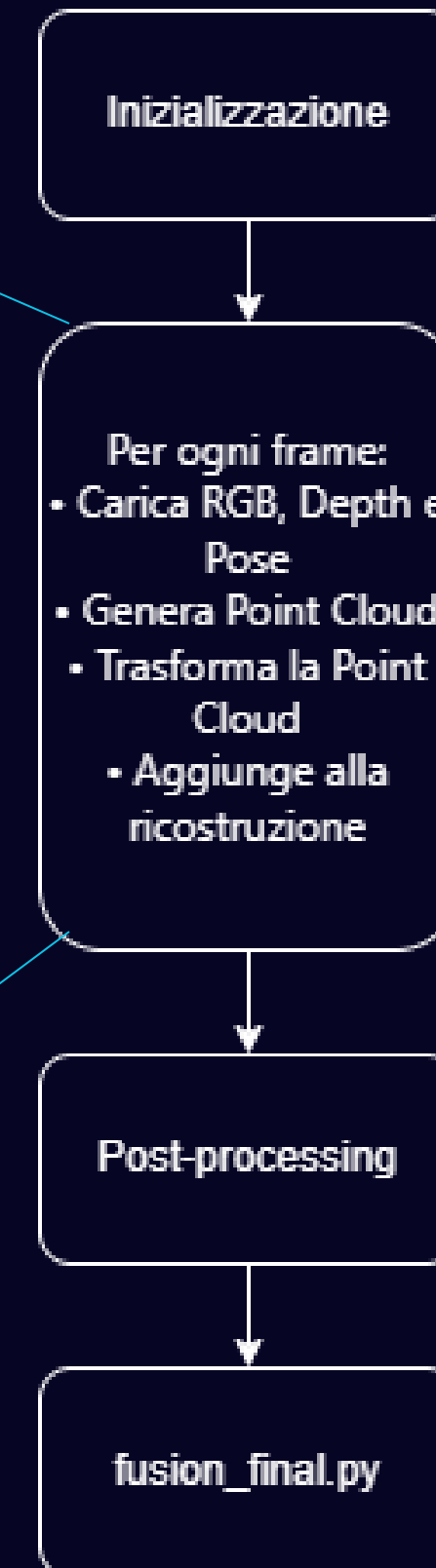
    # Creazione dell'immagine RGB-D per la ricostruzione 3D
    rgbd = o3d.geometry.RGBDImage.create_from_color_and_depth([
        color, depth,
        depth_scale=1.0,
        depth_trunc=10.0,
        convert_rgb_to_intensity=False
    ])

    # Generazione della nuvola di punti nel sistema di riferimento della camera
    pcd = o3d.geometry.PointCloud.create_from_rgbd_image(rgbd, intrinsic)

    # Trasformazione da s.d.r della camera a s.d.r del robot
    pcd.transform(T_cam_to_robot)

    # Trasformazione da s.d.r del robot a s.d.r di world
    T_robot_world = np.loadtxt(pose_path)
    pcd.transform(T_robot_world)

    # Aggiunta nuvola di punti locale a quella globale
    global_cloud += pcd
```

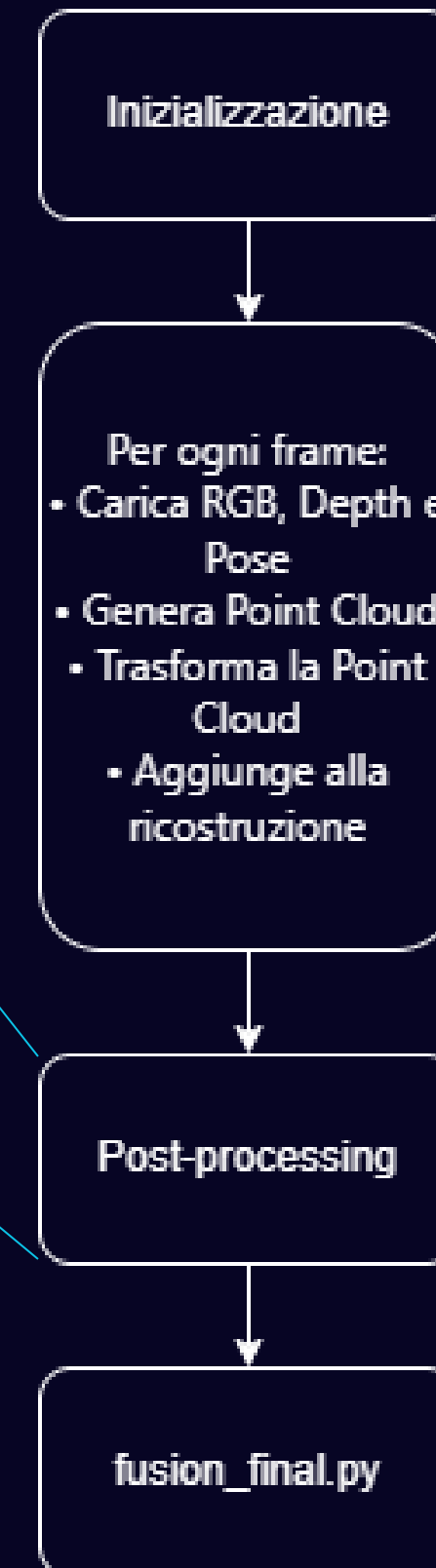


RICOSTRUZIONE 3D



```
# Elimina punti isolati dovuti a rumore nelle misure di profondità
global_cloud = global_cloud.voxel_down_sample(voxel_size=0.02)

global_cloud, _ = global_cloud.remove_statistical_outlier(
    nb_neighbors=10,
    std_ratio=2.0
)
```

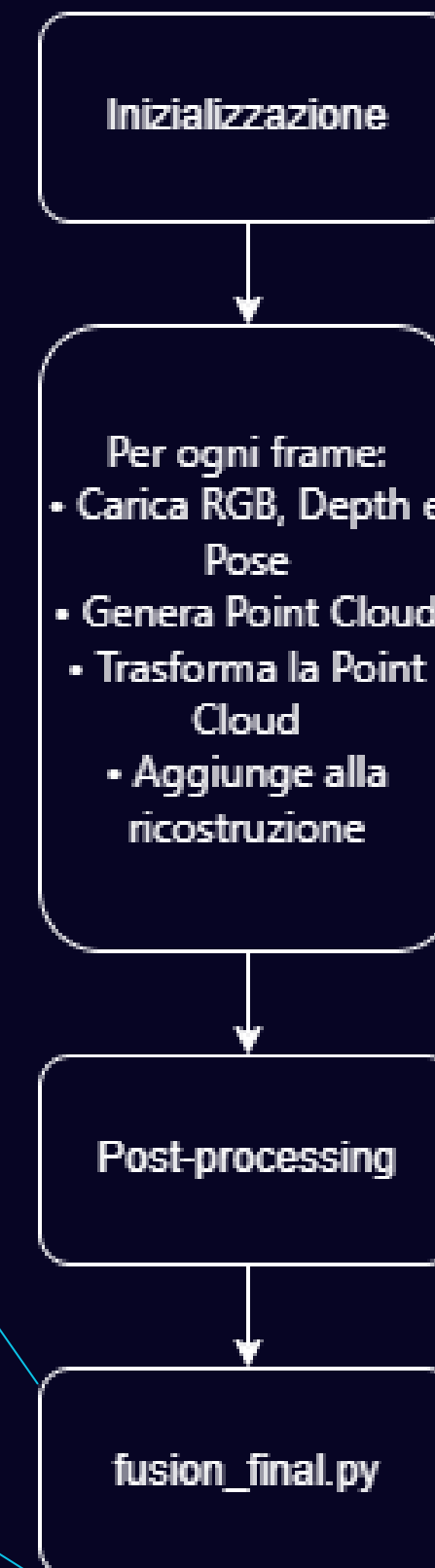


RICOSTRUZIONE 3D

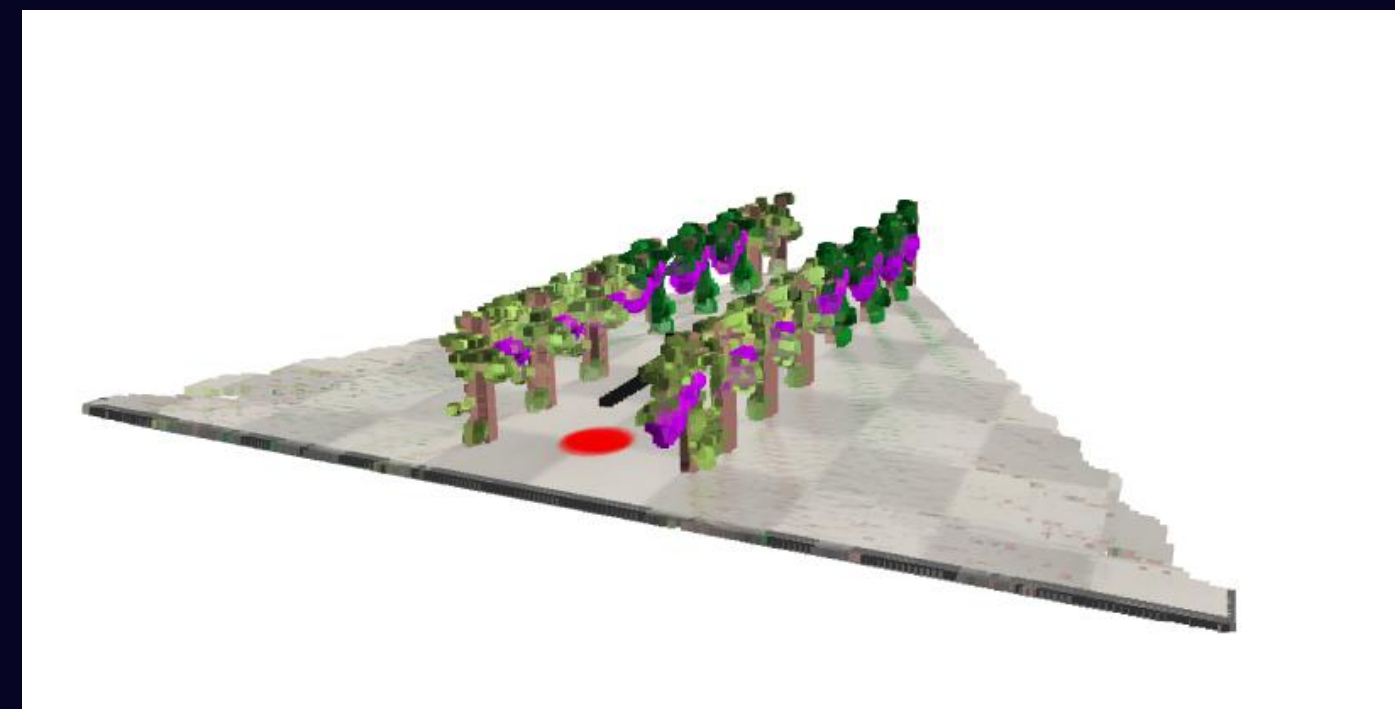
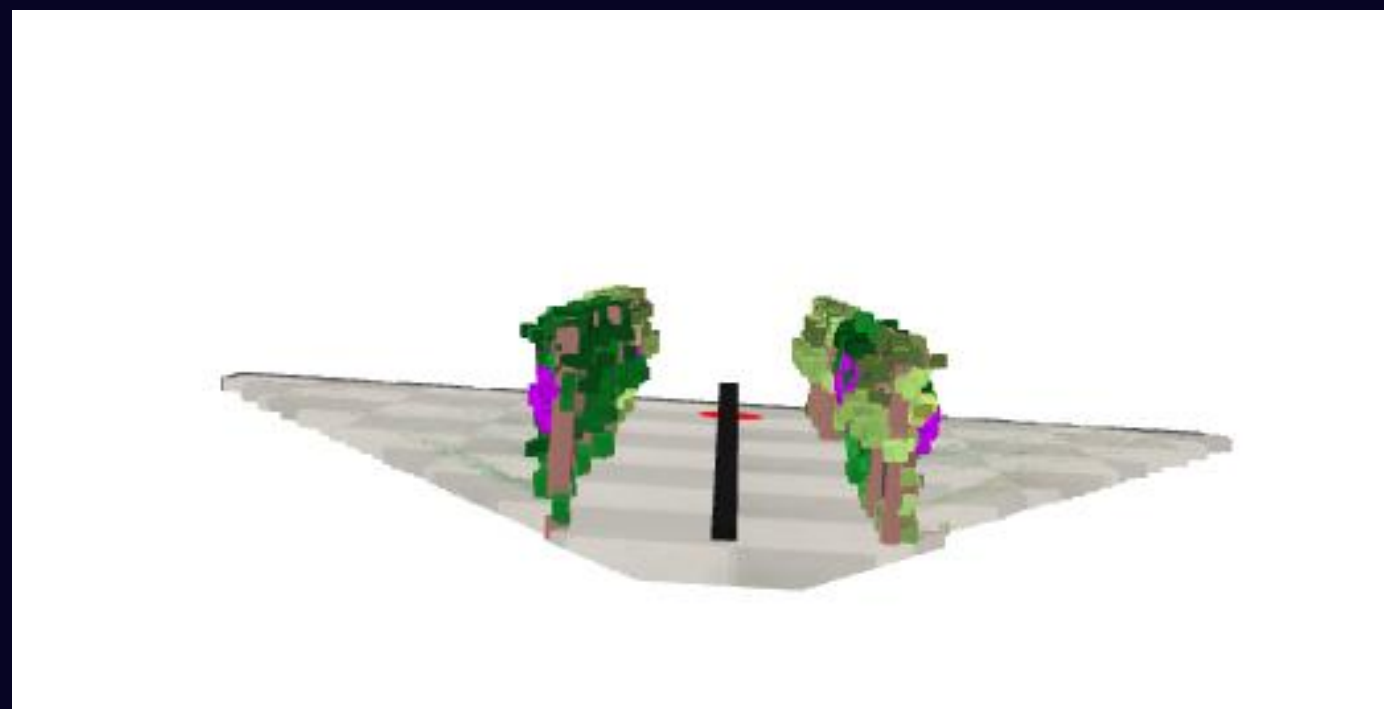
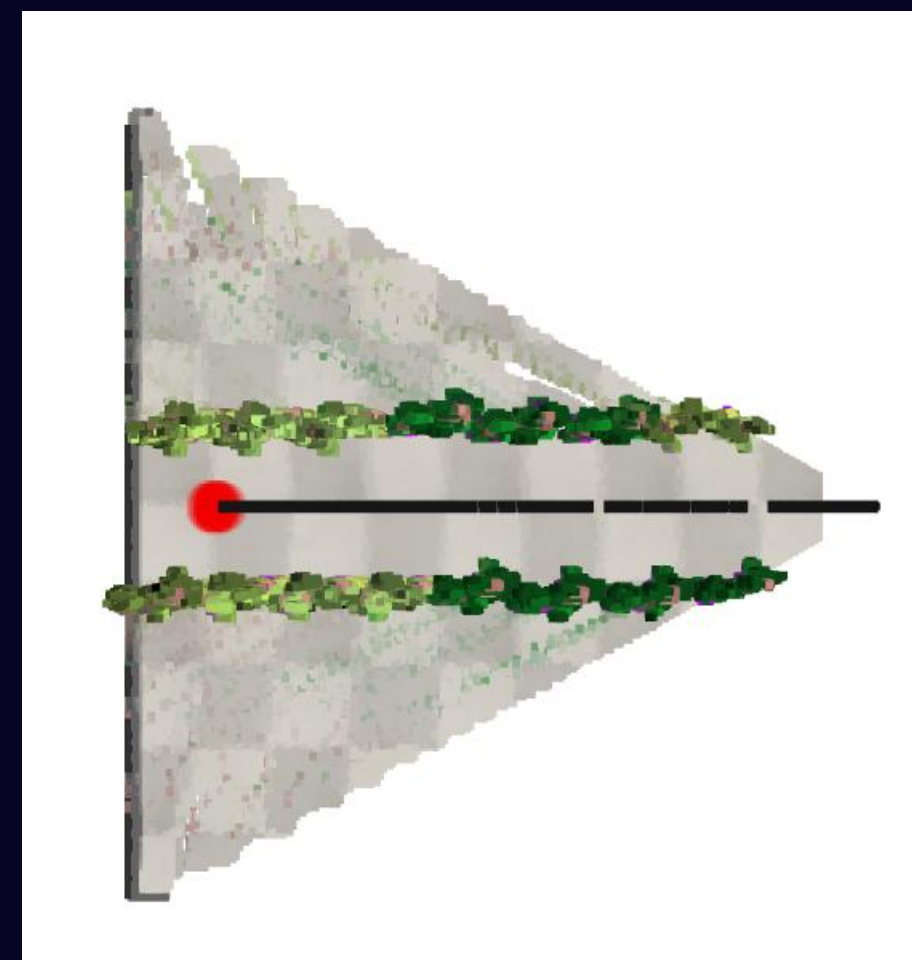
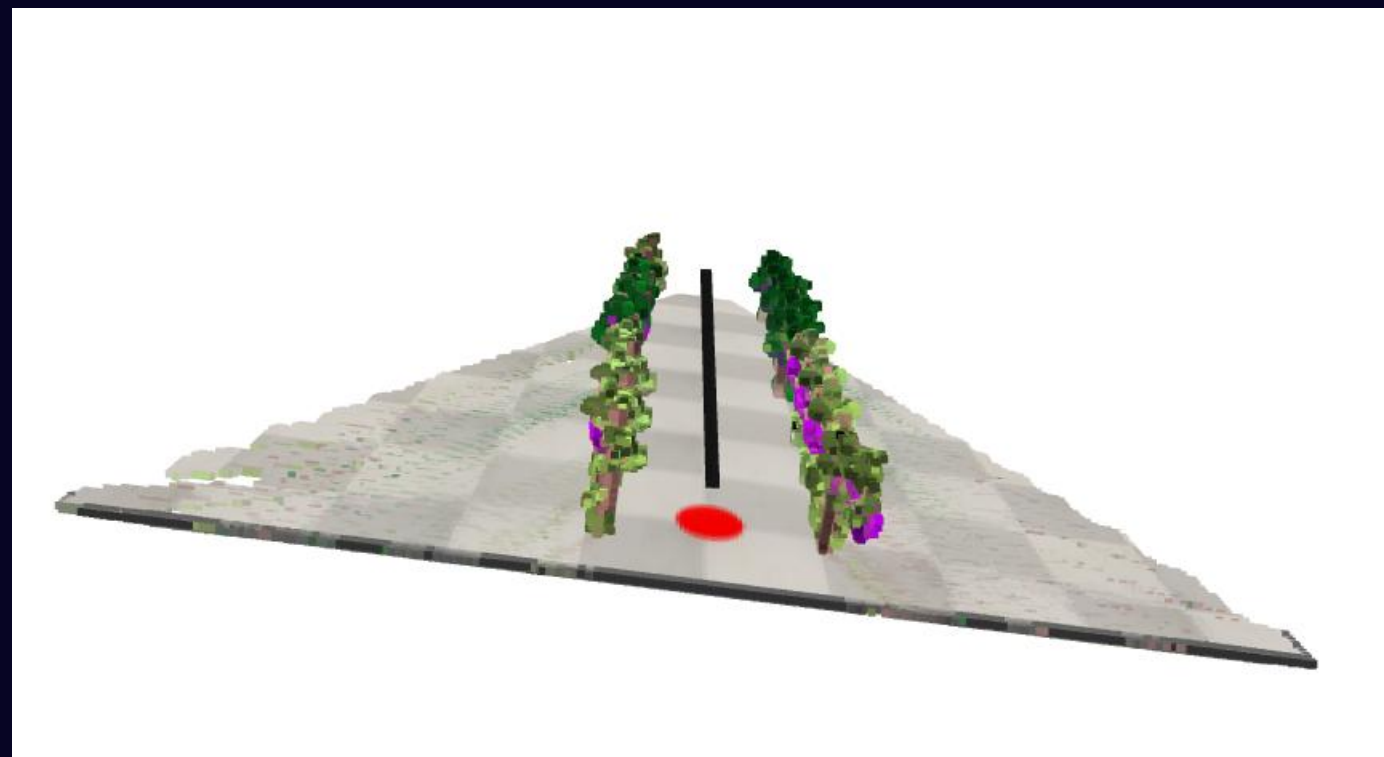


```
# Salva la nuvola di punti finale
o3d.io.write_point_cloud(OUT_FILE, global_cloud)
print(f"\nSalvato → {OUT_FILE}")

# Visualizza il risultato della fusione
o3d.visualization.draw_geometries(
    [global_cloud],
    window_name="Fusion Final",
    width=1280,
    height=720
)
```



RISULTATO



GRAZIE PER
L'ATTENZIONE